

AhamX Bodhi

Where I Becomes Infinite

Building Custom Tokenizers

Module 3.3: Tokenization — Coding

Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Build

- A BPE tokenizer from scratch in Python
- A custom tokenizer with HuggingFace Tokenizers
- A SentencePiece model on a custom corpus
- Tokenizer saving, loading, and integration with Transformers

Concept at a Glance

- **Duration:** 20 minutes
- **Bloom's Level:** B3 (Apply)
- **Difficulty:** L2 (Intermediate)
- **Language:** Python 3.10+
- **Libraries:** tokenizers, sentencepiece, transformers



Prerequisites

Before You Begin

- **Concepts:** What is Tokenization, BPE, WordPiece, SentencePiece (all covered earlier in this module)
- **Python:** Comfortable with classes, dicts, list comprehensions, file I/O
- **Libraries:** Basic familiarity with pip and virtual environments
- **Install:** `pip install tokenizers sentencepiece transformers datasets`
- **Hardware:** Any laptop/CPU is sufficient for all examples in this session





BPE From Scratch: Data Structures

What We Need to Track

- **vocab_freqs**: dict mapping each word (as tuple of chars) to its corpus frequency
- **merges**: ordered list of (A, B) merge rules produced during training
- **vocab**: set of all tokens currently in the vocabulary
- **get_stats()**: counts all adjacent pairs in the current vocab_freqs
- **merge_vocab()**: applies one merge rule to the entire vocab_freqs dict
- These four components are sufficient to implement a complete BPE tokenizer



BPE From Scratch: Initialize and Count Pairs

```
1  from collections import defaultdict
2
3  def get_vocab_freqs(corpus):
4      """Convert corpus list to char-tuple : frequency dict."""
5      vocab_freqs = defaultdict(int)
6      for word in corpus:
7          # Each word becomes tuple of chars + end marker
8          vocab_freqs[tuple(list(word) + ['</w>'])] += 1
9      return dict(vocab_freqs)
10
11 def get_stats(vocab_freqs):
12     """Count frequency of every adjacent pair."""
13     pairs = defaultdict(int)
14     for word_tuple, freq in vocab_freqs.items():
15         for i in range(len(word_tuple) - 1):
16             pairs[(word_tuple[i], word_tuple[i+1])] += freq
17     return pairs
```



BPE From Scratch: Merge and Train Loop

```

1  def merge_vocab(pair, vocab_freqs):
2      """Apply one merge rule to the vocabulary."""
3      new_vocab = {}
4      bigram = ' '.join(pair)
5      replacement = ''.join(pair)
6      for word_tuple, freq in vocab_freqs.items():
7          word_str = ' '.join(word_tuple)
8          word_str = word_str.replace(bigram, replacement)
9          new_vocab[tuple(word_str.split())] = freq
10     return new_vocab
11
12  def train_bpe(corpus, num_merges):
13     vocab_freqs = get_vocab_freqs(corpus)
14     merges = []
15     for _ in range(num_merges):
16         pairs = get_stats(vocab_freqs)
17         if not pairs:
18             break
19         best = max(pairs, key=pairs.get)
20         vocab_freqs = merge_vocab(best, vocab_freqs)
21         merges.append(best)
22     return merges, vocab_freqs

```


BPE From Scratch: Encoding at Inference

```

1 def encode(word, merges):
2     """Tokenize a single word using trained merge rules."""
3     tokens = list(word) + ['</w>']
4     for merge_pair in merges:
5         new_tokens = []
6         i = 0
7         while i < len(tokens):
8             if (i < len(tokens)-1 and
9                 tokens[i] == merge_pair[0] and
10                 tokens[i+1] == merge_pair[1]):
11                 new_tokens.append(''.join(merge_pair))
12                 i += 2
13             else:
14                 new_tokens.append(tokens[i])
15                 i += 1
16         tokens = new_tokens
17     return tokens
18
19 # Example
20 corpus = ['low', 'low', 'low', 'lower', 'lower', 'newest',
21           'newest', 'newest', 'widest']
22 merges, _ = train_bpe(corpus, num_merges=10)
23 print(encode('lowest', merges))
24 # ['<low', 'est</w>'] (if 'low' and 'est' were merged)

```





HuggingFace Tokenizers: Architecture

Four Pipeline Stages

- **Normalizer:** Unicode normalization, lowercasing, accent stripping (e.g., BertNormalizer)
- **Pre-tokenizer:** Initial split before the subword algorithm (e.g., Whitespace, ByteLevel)
- **Model:** The subword algorithm itself (BPE, WordPiece, or Unigram)
- **Post-processor:** Add special tokens, padding, truncation (e.g., BertProcessing)
- **Decoder:** Converts token IDs back to a readable string



HuggingFace: Train a BPE Tokenizer

```

1  from tokenizers import Tokenizer
2  from tokenizers.models import BPE
3  from tokenizers.trainers import BpeTrainer
4  from tokenizers.pre_tokenizers import Whitespace
5
6  # Step 1: Create tokenizer with BPE model
7  tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
8  tokenizer.pre_tokenizer = Whitespace()
9
10 # Step 2: Configure trainer
11 trainer = BpeTrainer(
12     vocab_size=30000,
13     special_tokens=["[UNK]", "[CLS]", "[SEP]",
14                   "[PAD]", "[MASK]"]
15 )
16
17 # Step 3: Train on files
18 files = ["train.txt"] # your corpus file
19 tokenizer.train(files, trainer)
20
21 # Step 4: Save
22 tokenizer.save("my_bpe_tokenizer.json")
23 print("Vocab size:", tokenizer.get_vocab_size())

```



HuggingFace: Encode and Decode

```

1  from tokenizers import Tokenizer
2
3  # Load saved tokenizer
4  tokenizer = Tokenizer.from_file("my_bpe_tokenizer.json")
5
6  # Encode a sentence
7  text = "Tokenization is fascinating!"
8  encoding = tokenizer.encode(text)
9
10 print("Tokens: ", encoding.tokens)
11 # -> ['Token', '##ization', 'is', 'fascinating', '!']
12 print("IDs: ", encoding.ids)
13 # -> [1234, 5678, 42, 891, 23]
14
15 # Decode back to string
16 decoded = tokenizer.decode(encoding.ids)
17 print("Decoded: ", decoded)
18 # -> "Tokenization is fascinating!"

```



HuggingFace: Byte-Level BPE (GPT-2 Style)

```
1 from tokenizers import Tokenizer
2 from tokenizers.models import BPE
3 from tokenizers.trainers import BpeTrainer
4 from tokenizers.pre_tokenizers import ByteLevel
5 from tokenizers.decoders import ByteLevel as ByteLevelDec
6
7 # Byte-level BPE: no language-specific pre-tokenizer
8 tokenizer = Tokenizer(BPE())
9 tokenizer.pre_tokenizer = ByteLevel(add_prefix_space=True)
10 tokenizer.decoder = ByteLevelDec()
11
12 trainer = BpeTrainer(vocab_size=50000,
13                     special_tokens=["<|endoftext|>"])
14 tokenizer.train(["corpus.txt"], trainer)
15 tokenizer.save("gpt2_style.json")
```





SentencePiece: Train from a Text File

```
1 import sentencepiece as spm
2
3 # Train BPE model with SentencePiece
4 spm.SentencePieceTrainer.train(
5     input='corpus.txt',
6     model_prefix='my_sp',          # outputs my_sp.model, my_sp.vocab
7     vocab_size=32000,
8     model_type='bpe',              # or 'unigram'
9     pad_id=0,
10    unk_id=1,
11    bos_id=2,
12    eos_id=3,
13    pad_piece='[PAD]',
14    unk_piece='[UNK]',
15    bos_piece='[BOS]',
16    eos_piece='[EOS]',
17    character_coverage=0.9995      # >0.9995 for multilingual
18 )
19 print("Training complete: my_sp.model")
```




SentencePiece: Encode, Decode, and Sample

```
1 import sentencepiece as spm
2
3 sp = spm.SentencePieceProcessor(model_file='my_sp.model')
4
5 # Standard encode (Viterbi / greedy)
6 tokens = sp.encode('Hello world', out_type=str)
7 print(tokens)    # ['_Hello', '_world']
8
9 ids = sp.encode('Hello world')
10 print(ids)       # [8325, 1362]
11
12 # Decode back
13 text = sp.decode(ids)
14 print(text)      # 'Hello world'
15
16 # Sampling (Unigram only -- use model_type='unigram')
17 samples = sp.sample_encode_as_pieces(
18     'tokenization', -1, 0.1) # nbest=-1, alpha=0.1
19 print(samples)    # probabilistic segmentation
```





Integration: Wrap as PreTrainedTokenizerFast

```
1  from transformers import PreTrainedTokenizerFast
2
3  # Wrap our custom HuggingFace tokenizer for Transformers API
4  fast_tokenizer = PreTrainedTokenizerFast(
5      tokenizer_file="my_bpe_tokenizer.json",
6      unk_token="[UNK]",
7      cls_token="[CLS]",
8      sep_token="[SEP]",
9      pad_token="[PAD]",
10     mask_token="[MASK]"
11 )
12
13 # Now works like any HuggingFace tokenizer
14 encoded = fast_tokenizer(
15     "Hello, my custom tokenizer!",
16     return_tensors="pt"
17 )
18 print(encoded["input_ids"])    # tensor([[...]])
19 fast_tokenizer.save_pretrained("./my_tokenizer_dir/")
```



Integration: Use Custom Tokenizer with a New Model

```
1 from transformers import (BertConfig, BertForMaskedLM,
2                             PreTrainedTokenizerFast)
3
4 tokenizer = PreTrainedTokenizerFast.from_pretrained(
5     "./my_tokenizer_dir/")
6
7 # Configure model to match tokenizer vocab size
8 config = BertConfig(
9     vocab_size=tokenizer.vocab_size,
10    hidden_size=256,
11    num_hidden_layers=4,
12    num_attention_heads=4,
13    intermediate_size=512
14 )
15 model = BertForMaskedLM(config)
16
17 total_params = sum(p.numel() for p in model.parameters())
18 print(f"Model parameters: {total_params:,}")
19 # Ready for training on your domain corpus
```



Evaluating Your Custom Tokenizer

Key Metrics

- **Fertility:** Average tokens per word (lower = more efficient)
- **OOV rate:** Fraction of words requiring [UNK]
- **Vocabulary coverage:** Fraction of corpus covered by top-K tokens
- **Encoding speed:** Tokens/second on your hardware

Target Values

- Fertility < 1.5 for domain vocabulary
- OOV rate $< 0.1\%$ with byte-level fallback
- Fast tokenizers: $> 1\text{M}$ tokens/second
- Compare fertility on held-out domain text vs general tokenizer



GenAI Application: Custom Domain Tokenizers

Why Build a Custom Tokenizer?

- **Medical:** Standard tokenizers split drug names like acetaminophen inefficiently; a medical-corpus tokenizer learns pharmaceutical morphology
- **Legal:** Legal Latin phrases and citation formats (e.g., *ibid.*, *supra* n.) tokenize poorly in general vocabularies
- **Code:** Programming language keywords, operators, and identifiers benefit from code-specific vocabularies (see Codex, StarCoder, DeepSeek Coder)
- **Low-resource languages:** Building a domain tokenizer for a local language not covered by general models

A domain-specific tokenizer with the same vocabulary size can reduce sequence lengths by 30–50% on domain text compared to a general tokenizer, directly reducing inference cost.



GenAI Application: Tokenizer Considerations in Fine-Tuning

What to Check Before Fine-Tuning

- Always use the **same tokenizer** as the base model — never swap tokenizers when fine-tuning
- If adding domain tokens (e.g., `<|medical|>`), use `tokenizer.add_tokens()` and resize model embeddings
- Set `tokenizer.pad_token = tokenizer.eos_token` for decoder-only models (GPT, Llama) before padding
- Use `max_length` and `truncation=True` consistently between training and inference
- Log token fertility on your fine-tuning dataset to verify the tokenizer is appropriate



Common Errors and How to Fix Them

Error	Cause	Fix
<code>KeyError: [UNK] not in vocab</code>	Special tokens not in BpeTrainer	Add to special_tokens list
<code>IndexError: input too long</code>	No max_length set	Add <code>truncation=True, max_length=512</code>
Padding fails in batch	No pad token set	Set <code>tokenizer.pad_token</code>
<code>UnicodeDecodeError</code>	Non-UTF8 corpus file	Re-encode corpus as UTF-8
Token IDs out of model vocab range	Tokenizer/model vocab mismatch	Call <code>model.resize_token_embeddings()</code>



Key Takeaways

What You Have Built and Learned

- A BPE tokenizer from scratch requires only a frequency dict, a pair counter, and a merge loop
- HuggingFace Tokenizers provides a production-grade pipeline: normalizer, pre-tokenizer, model, post-processor, decoder
- SentencePiece trains on raw text files with a single API call; use `model_type='bpe'` or `'unigram'`
- Custom tokenizers wrap into `PreTrainedTokenizerFast` for full Transformers API compatibility
- Domain-specific tokenizers reduce token fertility, cut API costs, and improve model performance on specialized text
- Always match the tokenizer to the model; resize embeddings when adding new tokens for fine-tuning

Thank You

Building Custom Tokenizers — Module 3.3